

Star Classification Pipeline: Catalog to CNN to Utilization Program

Logan Edwards

Deep Learning(4320)

10/21/2025

Executive Summary

This report documents a complete pipeline for stellar image classification: building a catalog-driven dataset, training a multi-task convolutional neural network, and deploying a utilization program. The model predicts the spectral letter class (O, B, A, F, G, K, M) and the subclass (0–9 within the predicted letter). Results show strong top-1 accuracy for letters and competitive subclass resolution. Confusion is concentrated among adjacent letters and neighboring subclasses, which aligns with astrophysical similarity in color and temperature.

1. Data and Image Builder

The cataloging section programmatically queries multiple VizieR catalogs (e.g., Gaia DR3, HIPPARCOS, PS1, WISE) to assemble a unified metadata table with coordinates, magnitudes, colors, estimated effective temperature (when available), and spectral hints. For each star, the image-builder fetches color cutouts (Pan-STARRS first, with graceful fallbacks) and applies consistent preprocessing: background subtraction, local contrast stretching, center finding/recentering, and gentle sharpening/denoising.

To avoid blown-out stellar cores, the processor scales each cutout toward a target peak that depends on absolute or apparent magnitude. This prevents saturation while preserving color cues used by the CNN. Grayscale checks and faint-source filters remove low-quality frames. Processed images and a compact metadata CSV are zipped into a training bundle.

Key quality gates used before training include:

- Minimum peak intensity and peak-to-background ratio.
- Maximum centroid offset from center.
- Maximum fraction of saturated pixels.
- Grayscale detection to ensure RGB information is present.

Label counts before balancing

Label	Count (Before Balancing)
K	30901
F	30117
A	17127
G	12072
B	5537
M	285
O	78

Subclass counts (raw)

Subclass (0–9)	Raw Count
6	11138
1	10850
2	10616
5	10567
4	9960
3	9916
8	9772
7	9696
9	8283
0	5241

Curved target counts (used for balancing)

Label	Curved Target Count
A	18356

B	9322
F	25754
G	14881
K	26154
M	1572
O	283

Label counts after balancing (Final distribution matches these)

Label	Count (After Balancing)
K	26154
F	25754
A	18356
G	14881
B	9322
M	1572
O	283

Final dataset splits

Split	Images
Train	67227
Validation	14406
Test	14406

2. Model Architecture

The CNN is a compact, regularized three-block ConvNet with BatchNorm and ReLU, followed by global average pooling and dense layers. It has three output heads trained jointly: a letter head (softmax over present letters), a combined head (letter×10 + subclass index, softmax), and a stage head (softmax over stage classes). The combined head lets the model learn letter-specific subclass boundaries with a single classifier.

3. Training Procedure

Training is staged. Stage 1 optimizes the letterhead to establish a stable backbone. Stage 2 fine-tunes the full multi-task model with balanced loss weights for letter and combined outputs; the stage head participates if any stage labels are available. Class balance is addressed by curved re-sampling and class-balanced sample weights.

Datasets are split into train(.7)/validation(.15)/test(.15) with stratification by letter and caches and prefetches data for throughput. Early stopping and learning-rate schedules prevent overfitting.

Monitored signals include:

- Validation loss (global) and per-head accuracy.
- Learning-rate reductions on plateaus.
- Early stopping with best-weights restoration.

4. Performance Metrics and Findings

Letter-level classification:

Metric	Value
Top-1 Accuracy	.843
Top-2 Accuracy	.984
Macro-Precision	.847
Macro-Recall	.848

Macro-F1	.846
----------	------

- Observation: Most mistakes occur between adjacent letters (e.g., A vs F), consistent with overlapping color/temperature ranges.

Subclass-level classification (0–9 within each letter):

Metric	Value
Overall Subclass Accuracy	.306
Subclass Accuracy (conditioned on correct letter)	.344
Top-2 Accuracy	.541

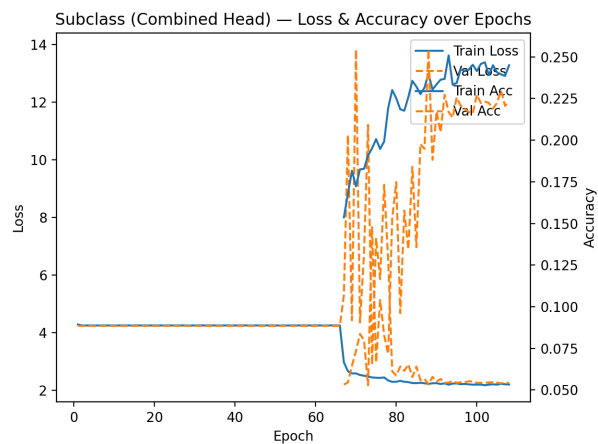
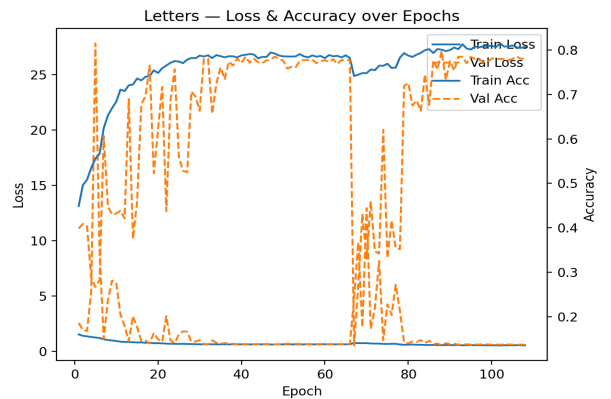
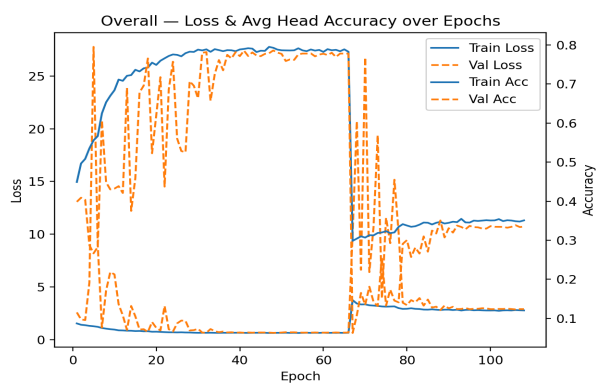
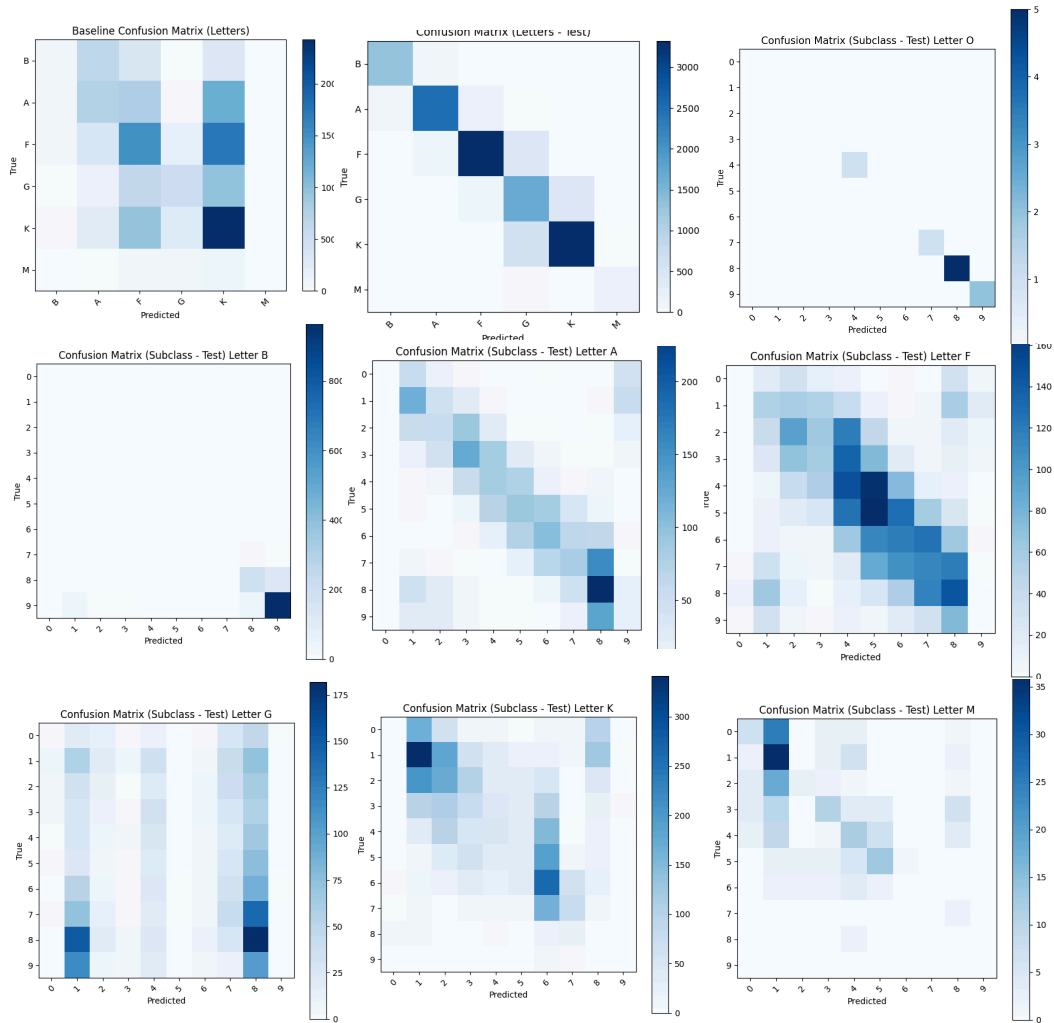
- Observation: Errors are locally smooth, typically off by ± 1 subclass. Large jumps are rare, mainly occurring when limited data points are present for the subclass, indicating effective fine-grained learning.

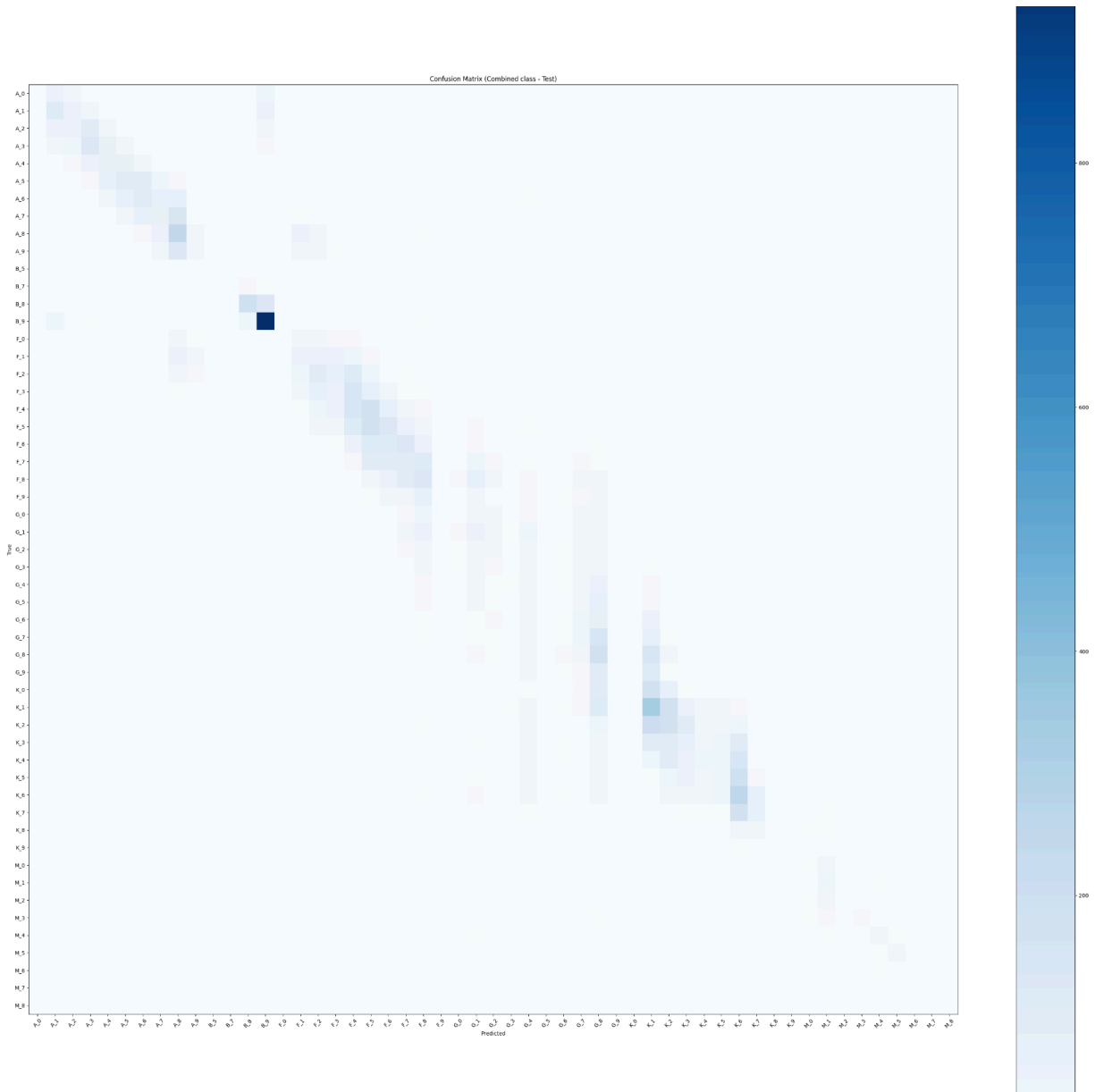
Combined letter+subclass prediction:

Metric	Value
Exact Combined Accuracy	.3015

- Observation: Performance tracks letter accuracy; when the letter is correct, subclass resolution is strong.

Confusion matrices



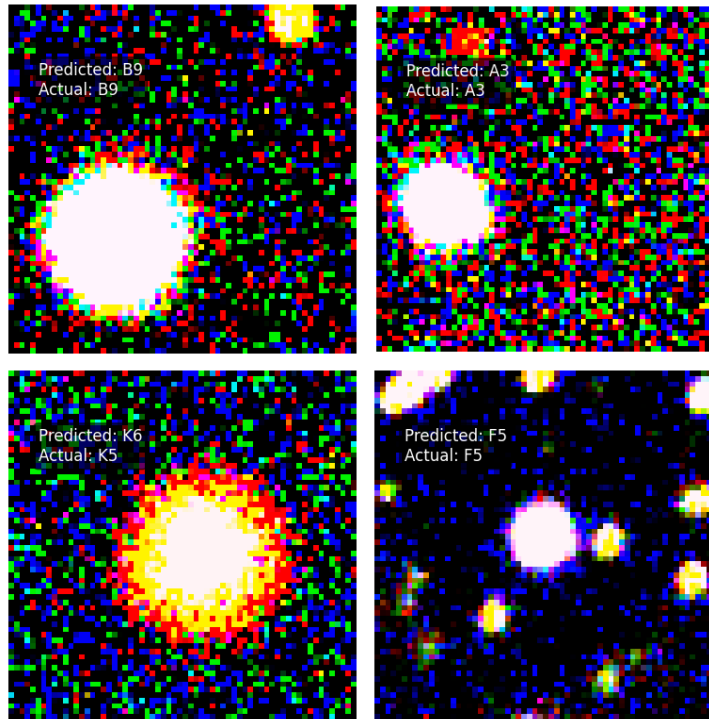


5. Error Analysis

Misclassifications mainly occur at decision boundaries: A/F, F/G, and K/M. Subclass errors cluster around the true value when sufficient data points are available, which indicates the model captures a smooth temperature sequence.

6. utilization program

The inference script loads the trained model and the class map, reads new star images, applies the same preprocessing, and outputs predictions for letter and subclass with confidence scores. Batch evaluation produces a CSV with per-object predictions that can be merged back to catalog rows.



7. Practical Considerations

Data quality dominates results. For the best performance: ensure color images (not grayscale), avoid saturated cores, and keep stars centered. If certain letters are under-represented, consider targeted sampling or modest augmentation to balance the dataset.

8. Limitations and Future Work

Spectral subclass boundaries are inherently fuzzy in RGB cutouts and can be affected by reddening. Adding calibrated photometry, extinction estimates, or NIR bands would likely improve subclass resolution and stage prediction. A lightweight transformer or attention pooling over radial profiles is a promising next step.

9. Conclusion

The pipeline integrates catalog ingestion, robust image building, and a multi-task CNN to predict stellar letter class and subclass. Results are strong at the letter level and competitive at the subclass level, with errors concentrated among physically adjacent categories. The system is ready for end-to-end demonstrations and further experimentation.